

110.12. La fonction poserDevant() — une case qui passe devant les lutins

#include <poserDevant> : `poserDevant(colonne, ligne, tuile, palette)` pose une tuile ET la fait passer devant les personnages (Game Boy Color).

1. Ce qu'il faut comprendre

Ce qu'elle fait : elle pose une tuile, comme `poser()`, et la met dans une palette **en la faisant passer devant les lutins**, comme `teindre(colonne, ligne, palette | DEVANT)`. Deux lignes en une.

La ligne à écrire : **#include <poserDevant>**. Pas besoin d'inclure aussi `poser` ni `teindre` : la console s'en charge.

Ses arguments : la colonne, la ligne, la tuile, la palette (0 à 7). Comme pour `teindre()`, la case ne cache le lutin que là où elle est **dessinée** (1, 2 ou 3) : ses 0 laissent voir le lutin.

Ce qu'elle coûte : **rien de plus** que `poser()` suivi de `teindre()`. Le compilateur remplace la ligne par ces deux-là, mot pour mot : la cartouche a exactement les mêmes octets.

Essaie : pose le héros 1 sur le mur, lui aussi (`sprite(1, 72, 64, HEROS)`) : il disparaît.

On retrouve poserDevant dans 1 leçon

[110.13](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **325 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	103 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	32 o	← CopierTuiles — 0 lettre sur 43, et 2 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

sprite()	× 2	46 o sur place, dans main()
couleurFond()	× 2	24 o sur place, dans main()

```

teindre()    × 1    17 o sur place, dans main()
poser()      × 1    6 o sur place, dans main()
image()      × 1    6 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- #include <poserDevant> : une case qui passe devant les lutins ----
// Un mur posé avec poserDevant(). Le héros 0, posé dessus, disparaît derrière ;
// le héros 1, à côté, se voit.

#include <Tuile>
#include <couleurFond>
#include <sprite>
#include <poserDevant> // poserDevant() : poser + teindre(..., DEVANT), en une ligne

```

2. UN DESSIN — MUR

```

Tuile MUR = {
    // des briques (1), des joints (3) : aucun 0
    "33333333", "11131111", "11131111", "33333333",
    "13111113", "13111113", "33333333", "11131111",
};

```



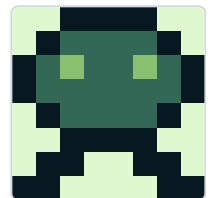
à l'écran

3. UN DESSIN — HEROS

```

Tuile HEROS = {
    // 0 = transparent
    "00333300", "03222230", "32122123", "32222223",
    "03222230", "00333300", "03300330", "33000033",
};

```



à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    couleurFond(2, 1, 26, 12, 6); // palette 2 : la brique
    couleurFond(2, 3, 10, 4, 2);  // et ses joints
    poserDevant(9, 8, MUR, 2);    // le mur, case (9, 8), palette 2, DEVANT les lutins

    sprite(0, 72, 64, HEROS);     // sur le mur (72 = 9 × 8, 64 = 8 × 8) : caché
    sprite(1, 40, 64, HEROS);     // à côté : visible

    while (true) {
        image();
    }
}

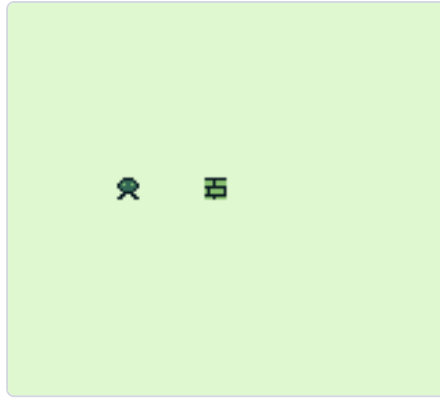
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un mur de briques et un héros à sa gauche. L'autre héros, posé sur le mur, ne se voit pas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la case (9, 8) : palette 2, et DEVANT (130)
- ✓ sur le mur, on voit la brique, pas le héros (26/12/6)

Ensuite : **110.13. Devant, derrière : le buisson sur Game Boy Color, poserDevant()** — chapitre 17 : Le mouvement : les lutins au pixel près. · gameboy3 — écrire une cartouche Game Boy en C++